NESTJS REQUEST FLOW
Контролери, DTO, параметр-декоратори, Pipe, Interceptor

Заняття №7

Node.js Pro

Лектор
Кирило Орлов

r_d

ОДИН ЗАПИТ — УСЯ ЛЕКЦІЯ



- Герой: POST /users {"name":"Ada", "age":"тридцять", "admin":true, ...}
- маршрут — збіг за метаданими @Controller + @Post
- аргументи — @Body/@Param кажуть, звідки брати
- контракт — DTO + Pipe: пропустити або 400
- метод — готові аргументи, без req і res
- Interceptor — бачить і вхід, і вихід
- відповідь — повернене значення → JSON

ЩО РОБИТЬ ДИСПЕТЧЕР — ШІСТЬ КРОКІВ

01. Маршрут

префікс @Controller + шлях @Get → повний path

02. Збіг

диспетчер шукає метод за method + path, дістає :params

03. Метадані параметрів

мапа {індекс → звідки брати}, записана декораторами

04. Pipe

кожен аргумент проходить трансформацію й валідацію

05. Виклик

метод отримує готові аргументи — не req, не res

06. Серіалізація

повернене значення стає JSON-відповіддю автоматично

СТАНЦІЯ 1 · МАРШРУТ ЖИВЕ В МЕТАДАНИХ

```
// 01-controller.ts — маршрут у метаданих
import { Body, Controller, Get, Param, Post }
  from '@nestjs/common';

@Controller('users')          // префікс → КЛАС
class UsersController {
  @Post()                     // POST /users → МЕТОД
  create(@Body() body: any) { // без DTO — сирий JSON
    return { saved: body, ageType: typeof body.age };
  }

  @Get(':id')                 // GET /users/:id
  findOne(@Param('id') id: string) {
    return { id, idType: typeof id };  // завжди string
  }
}

// тіло розпарсив НЕ Nest, а express.json(),
// який NestFactory вмикає сам — шар невидимий

// $ curl -X POST localhost:20700/users \
//        -H 'content-type: application/json' \  ← інакше не JSON
//        -d '{"name":"Ada","email":"ada@x.dev",
//             "age":"тридцять","admin":true}'
// → 201 {"saved":{…,"age":"тридцять","admin":true},
//        "ageType":"string"}     ← ніхто не перевірив
```

Декоратор нічого не робить — він записує

1 · @Controller('users') кладе префікс на клас

2 · @Get(':id') кладе метод і шлях на функцію

3 · @Param('id') кладе {індекс 0 → param 'id'}

GET /users/42 → метод отримує '42' першим аргументом

урок: req і res у метод не приходять взагалі

СТАНЦІЯ 2 · ЗВІДКИ БЕРЕТЬСЯ КОЖЕН АРГУМЕНТ

Декоратор	Звідки бере	Тип на виході
@Body()	тіло запиту, вже розпарсене	об ′єкт; із pipe — екземпляр DTO
@Param('id')	сегмент шляху за назвою	завжди рядок — число робить pipe
@Query('limit')	query-string	рядок або масив рядків
@Headers('x-id')	заголовок запиту	рядок або undefined
власний	createParamDecorator((data, ctx) => ...)	що завгодно: @CurrentUser()

```
// 04-param-decorator.ts — власні параметр-декоратори через createParamDecorator
import { createParamDecorator, ExecutionContext } from '@nestjs/common';
type SessionUser = { id: string; name: string; role: 'admin' | 'member' };
const UserAgent = createParamDecorator((_d: unknown, ctx: ExecutionContext) =>
  ctx.switchToHttp().getRequest<Request>().headers['user-agent']);  // ось і все
const CurrentUser = createParamDecorator(   // data — те, що в дужках декоратора
  (data: keyof SessionUser | undefined, ctx: ExecutionContext) => {
    const req = ctx.switchToHttp().getRequest<Request>();
    const id = req.headers['x-user-id'];   // у проді сюди кладе auth-guard (L#8)
    if (typeof id !== 'string') return null;
    const user = { id, name: `user-${id}`, role: id === '1' ? 'admin' : 'member' };
    return data ? user[data] : user;       // @CurrentUser('role') → одне поле
  });

@Get()  // хендлер не знає ні про headers, ні про request
me(@CurrentUser() user, @CurrentUser('role') role, @UserAgent() ua) {
  return { user, role, ua };
}
// $ curl -H 'x-user-id: 1' :20700/me
// → {"user":{"id":"1","name":"user-1","role":"admin"},"role":"admin",
//    "ua":"curl/8.7.1"}  · без заголовка x-user-id → {"user":null,…}
```



СТАНЦІЯ 3 · У @BODY ПРИЇХАЛО ВСЕ, ЩО ПРИСЛАЛИ

age: "тридцять" — рядок спокійно їде в «БД»: ageType = "string"

admin: true — поле, якого ніхто не просив, збережено

@Param('id') — завжди рядок: він вирізаний з URL

перевірки або копіюються по обробниках, або їх немає



Рішення — контракт у ТИПІ: DTO описує вхід, Pipe перевіряє його ДО хендлера — один раз для всіх маршрутів.

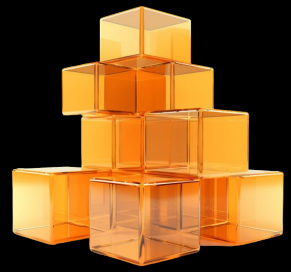
DTO: КОНТРАКТ ВХОДУ, А НЕ «ЩЕ ОДИН ІНТЕРФЕЙС»

- клас, а не interface — інтерфейс у рантаймі не існує
- декоратори полів описують правила: @IsEmail, @Min, @IsOptional
- whitelist: true — зайві поля відрізаються, а не проходять далі
- forbidNonWhitelisted — зайве поле стає помилкою 400
- transform: true — у метод приходить ЕКЗЕМПЛЯР, а не plain-об ′єкт
- той самий DTO потім згенерує Swagger і типізованого клієнта

ОДИН КЛАС — ДВІ РОЛІ ОДРАЗУ

```
class CreateUserDto {
  @IsString() @MinLength(2)
  name!: string;
  …
}
```

- для КОМПІЛЯТОРА: тип (зникне після tsc)
- для РАНТАЙМУ: правила в метаданих класу — їх і читає ValidationPipe



«БЛАНК ЗАЯВИ»: DTO ЯК АНАЛОГІЯ

ЧАСТИНА NEST

- DTO-клас
- @IsEmail() над полем
- ValidationPipe
- 400 зі списком полів
- обробник отримує екземпляр DTO

АНАЛОГІЯ (КАНЦЕЛЯРІЯ)

- бланк із назвами граф
- вимога до графи: «тільки пошта»
- приймальне вікно, що перевіряє бланк
- бланк повернули: графи підкреслено
- далі йде вже перевірений бланк, а не аркуш у клітинку

Де аналогія ламається: бланк перевіряє людина й може «закрити очі». Pipe — ні: він або пропускає, або відхиляє. І він же може ЗМІНИТИ бланк — привести '42' до числа 42.





```
// 02-dto.ts — DTO це КЛАС: типи TS стираються компіляцією, декоратори лишаються
import { IsEmail, IsInt, IsString, Min, MinLength } from 'class-validator';
class CreateUserDto {
  @IsString() @MinLength(2)  name!: string;
  @IsEmail()                 email!: string;
  @IsInt() @Min(16)          age!: number;
}

@Post() @UsePipes(new ValidationPipe())          // дефолт: 400 є, але admin пролазить
create(@Body() dto: CreateUserDto) { return { saved: dto, className: … }; }  // "Object"

@Post('strict') @UsePipes(new ValidationPipe({ whitelist: true, transform: true }))
createStrict(@Body() dto: CreateUserDto) { … }  // admin зрізано, className CreateUserDto

@Post('coerce') @UsePipes(new ValidationPipe({ whitelist: true, transform: true,
  transformOptions: { enableImplicitConversion: true } }))     // ← коерсія: «30» стане 30
createCoerce(@Body() dto: CreateUserDto) { return { ageType: typeof dto.age }; }

// $ curl -X POST localhost:20700/users -H 'content-type: application/json' \
//        -d '{"name":"A","email":"nope","age":15}'         ← БЕЗ -H тіло НЕ JSON
// → {"message":["name must be longer than or equal to 2 characters","email must be an
//    email","age must not be less than 16"],"statusCode":400} [400]   ← УСІ порушення
// $ POST /users/strict {"age":"30"} → 400 · /users/coerce {"age":"30"} → 201 ageType number
```

ПАСТКА, НА ЯКІЙ ЗАСТРЯГАЮТЬ УСІ

ValidationPipe() без опцій
400 на невалідне працює, але admin пролазить, а dto — plain Object

whitelist: true + transform: true
admin зрізано, у метод їде екземпляр CreateUserDto

збірка без emitDecoratorMetadata
той САМИЙ невалідний запит → 201: мовчки зникла валідація, а з нею й whitelist

Чому так
ValidationPipe дізнається клас у @Body() з метаданих design:paramtypes — їх пише tsc із emitDecoratorMetadata. esbuild/tsx прапорця не вміють: збірка пройде, а той самий запит поверне 201 {"className":"Object"} замість 400 — мовчки. Опції пайпа — не тюнінг, а політика контракту: whitelist — доля зайвих полів, transform — у якому вигляді дані заходять у метод.

PIPE: ПЕРЕТВОРИТИ АБО ВІДХИЛИТИ — ОДИН АРГУМЕНТ

працює з ОДНИМ аргументом, не з усім запитом

виконується безпосередньо перед викликом обробника

два режими: transform (рядок → число) і validate (кинути 400)

Вбудовані: ParseIntPipe, ParseUUIDPipe, ValidationPipe. Свій — клас із transform(). Рівні: глобально → контролер → метод → @Param.

МІСТОК ДО L#4

Правило коерсії у Fastify живе в ОПИСІ РОУТА: schema.params робить «42» числом. У Nest це окремий об ′єкт: перевикористати, підмінити в тестах, повісити глобально. Ціна — поставити його треба самому. Прогін лежить у репозиторії L#4 (крок 09-fastify-tour.mjs) — в ефірі його не було.



```
// 03-custom-pipe.ts — увесь «вбудований» ParseIntPipe: ось стільки коду
@Injectable()
class ParsePositiveIntPipe implements PipeTransform<string, number> {
  transform(value: string, metadata: ArgumentMetadata): number {
    const n = Number(value);
    if (!Number.isInteger(n) || n <= 0) {
      throw new BadRequestException(          // виняток у пайпі = 400
        `«${value}» не є додатним цілим (${metadata.type} '${String(metadata.data)}')`,
      );
    }
    return n;                                 // це і стане аргументом
  }
}

@Get('raw/:id')     raw(@Param('id') id: string) { … }
@Get('builtin/:id') builtin(@Param('id', ParseIntPipe) id: number) { … }
@Get('custom/:id')  custom(@Param('id', ParsePositiveIntPipe) id: number) { … }  // ← КЛАС

// $ curl :20700/users/raw/abc     → 200 {"id":"abc","type":"string"}   ← пролізло!
// $ curl :20700/users/builtin/abc → 400 Validation failed (numeric string is expected)
// $ curl :20700/users/custom/-5   → 400 «-5» не є додатним цілим (param 'id')
```

СТАНЦІЯ 4 · ВІД HTTP-ЗАПИТУ ДО ВИКЛИКУ МЕТОДУ

```
POST /users   {"name":"Ada","email":"ada@x.dev","age":30,"admin":true}
    ↓
маршрут           @Controller('users') + @Post()  →  збіг
    ↓
метадані методу   {0: {type:'body'}}          ← записав @Body()
    ↓
сирі значення     [ {name, email, age, admin} ]
    ↓
PIPE              ValidationPipe({ whitelist: true, transform: true })
    ↓                       ↓ невалідне тіло
    ↓        400 {"message":[…всі порушення…],"error":"Bad Request"}
    ↓
аргументи         [ CreateUserDto{name, email, age} ]   ← admin зрізано
    ↓
ВИКЛИК            create(dto)         ← req і res сюди НЕ приходять
    ↓
повернене значення  →  JSON + 201
    ↓
INTERCEPTOR (after)     → обгорне: { data, tookMs }
```



ЧЕКПОЙНТ: ДЕ ЗАРАЗ НАШ ЗАПИТ

Пройдено — дорога туди	Попереду — дорога назад
маршрут знайдено за метаданими @Controller + @Post	Interceptor — обгортка навколо виклику методу
аргументи зібрано: мапа {індекс → звідки брати}	відповідь: повернене значення → JSON
тіло пройшло контракт: DTO + ValidationPipe або 400	той самий контракт на zod — друга школа валідації
метод отримав готові аргументи — без req і res	фінальна таблиця: middleware vs pipe vs interceptor

ДРУГА ПОЛОВИНА
Запит уже всередині методу — далі дорога назад: обгортка виклику й відповідь. Тримається все на тих самих метаданих із L#6: interceptor — такий самий клас у контейнері, як і сервіси.

СТАНЦІЯ 5 · INTERCEPTOR: ОБГОРТКА НАВКОЛО ВИКЛИКУ

01. Код до виклику, виклик, код після — в одному місці

02. Типове: заміряти час, залогувати, закешувати

03. Може ЗМІНИТИ відповідь — на відміну від guard

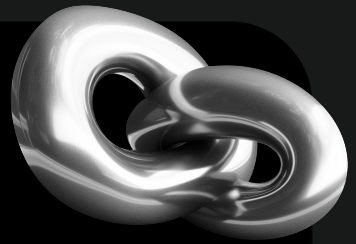
04. Працює на RxJS: next.handle().pipe(map(...), tap(...))

05. Ставиться глобально, на контролер або на метод — як і pipe

```
запит ──▶ [ interceptor ДО ] ──▶ хендлер ──▶ [ interceptor ПІСЛЯ ] ──▶ відповідь
          started = Date.now()    спить 120 мс    map(): { data, tookMs: 121 }
          бачить req і body       повертає {id:42}    лог: «за 121 мс»
місток до L#4: в іменованому циклі Fastify «до» і «після» — ДВА РІЗНІ хуки, зв'язати їх
можна лише полем у request. Interceptor — ОДНА функція: started живе в замиканні до map().
```

«МИТНИЦЯ»: PIPE ПРОТИ INTERCEPTOR

ЧАСТИНА NEST	АНАЛОГІЯ (АЕРОПОРТ)
• Pipe • Interceptor • Pipe кидає помилку • Interceptor міняє відповідь • обробник	• митний контроль конкретної валізи • супровідник, що йде з тобою туди й назад • валізу не пропустили — далі не йдеш • супровідник переклав речі на видачі • стійка, де тобі щось видають



```
// 05-interceptor.ts — обгортка бачить І вхід, І вихід хендлера
@Injectable()
class TimingInterceptor implements NestInterceptor {
  intercept(ctx: ExecutionContext, next: CallHandler): Observable<unknown> {
    const req = ctx.switchToHttp().getRequest<Request>(), started = Date.now();  // ── ДО
    console.log(`[interceptor] → вхід:  ${req.method} ${req.url}`);  // + body
    return next.handle().pipe(    // next.handle() — виклик хендлера в Observable
      map((data) => {                                    // ── ПІСЛЯ хендлера
        const tookMs = Date.now() - started;
        console.log(`[interceptor] ← вихід: ${JSON.stringify(data)} за ${tookMs} мс`);
        return { data, tookMs };                         // ← обгорнули відповідь
      }));
  }
}
@Post() @UseInterceptors(TimingInterceptor)   // ← вішаємо на метод (можна й глобально)
async create(@Body() b: {name?:string}) { await sleep(120); return {id:42, name:b.name}; }

// $ curl localhost:20700/users -H 'content-type: application/json' -d '{"name":"Ada"}'
// → {"data":{"id":42,"name":"Ada"},"tookMs":121}   ← 120 мс сну + накладні, число плаває
// лог: → вхід:  POST /users body={"name":"Ada"}  ·  ← вихід: {…} за 121 мс
```

class-validator ЧИ zod: ДВА ЖИВІ ПІДХОДИ

class-validator — правила декораторами прямо на полях класу

нативний для Nest: ValidationPipe працює з коробки

потребує class-transformer і живого класу в рантаймі

zod — схема окремо, тип виводиться з неї через z.infer

одне джерело правди: змінив схему — змінився тип

⚠ zod 4 змінив API помилок: error.issues замість error.errors

ХТО КОМПІЛЮЄ КОНТРАКТ, А ХТО ХОДИТЬ ПО МЕТАДАНИХ

На L#4 звучало: ajv компілює перевірку зі схеми. Zod 4 робить те саме — z.object збирає «fastpass», згенеровану функцію, де ключі вписані просто в код. class-validator не генерує нічого: він щоразу обходить MetadataStorage. Кодогенерація — швидкість, метадані — гнучкість. Прогін: npm run who-compiles.

```
// 06-zod.ts — той САМИЙ контракт: схема замість класу з декораторами
import { z, ZodType } from 'zod';
const CreateUserSchema = z.object({
  name: z.string().min(2),
  email: z.email(),                    // Zod 4: z.email(), НЕ z.string().email()
  age: z.int().min(16),
});
type CreateUserDto = z.infer<typeof CreateUserSchema>;  // тип ВИВОДИТЬСЯ зі схеми
class ZodValidationPipe implements PipeTransform {
  constructor(private readonly schema: ZodType) {}
  transform(value: unknown) {
    const result = this.schema.safeParse(value);
    if (!result.success) throw new BadRequestException({ statusCode: 400,
      error: 'Bad Request',                // Zod 4: error.issues, було error.errors
      message: result.error.issues.map((i) => `${i.path.join('.')}: ${i.message}`) });
    return result.data;                    // зайві ключі вже зрізані strip-дефолтом
  }
}
// $ POST {"name":"A","email":"nope","age":15} → 400 {"message":["name: Too small:
//   expected string to have >=2 characters","email: Invalid email address",…]}
// $ POST {"age":"30"} → 400 "age: Invalid input: expected number, received string"
```

PIPE, INTERCEPTOR, MIDDLEWARE: ХТО ЗА ЩО

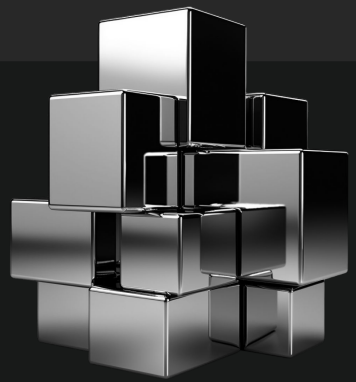
Сутність	Що бачить	Що може
Middleware	сирі req / res, про контролер не знає	змінити req, урвати запит
Pipe	ОДИН аргумент і його метадані	перетворити значення або кинути 400
Interceptor	виклик обробника цілком: до і після	змінити відповідь, заміряти, закешувати
Порядок	middleware → interceptor(before) → pipe → handler	pipe — найближче до обробника
Guard і Filter	«пускати чи ні» та «що робити з помилкою»	повний цикл — наступна лекція

ЩО СВІДОМО ЛИШАЄМО НА L#8

Guards і Exception filters сьогодні лише назвали. Повний цикл запиту — окрема лекція: це топ-1 питання минулого потоку.

AsyncLocalStorage і request-id — теж L#8. Сьогодні аргументи приходять явно; далі — як передати контекст, куди аргумент не доходить.

⚠ Serialization (@Exclude на passwordHash) — детально у L#9 з дизайном API. Ефект той самий, що schema.response у Fastify на L#4.



ПІДСУМКИ + МІСТ ДО L#8

КОНТРОЛЕР

маршрут і аргументи живуть у метаданих; req/res у метод не приходять

DTO

клас із правилами; без emitDecoratorMetadata валідація мовчки зникає

PIPE / INTERCEPTOR

pipe — один аргумент, interceptor — увесь виклик разом із відповіддю

ДЗ #7

8 балів · дедлайн 20.08 · mini-Nest: @Controller, @Get/@Post, @Body/@Param/@Query, диспетчер поверх node:http, DTO-валідація через pipe. Частина 2 з 3

ДАЛІ

L#8 — повний цикл запиту й AsyncLocalStorage

Q&A

???

r_d

ЗАВЖДИ Є КУДИ ЗРОСТАТИ